Personal Programming Project Report

Title: Misinformation Detector: A Machine Learning Approach to Fake News Classification

1. **(5pts) Honor Code and LLM Usage for this Report.**

Honor Code: I have neither given nor received unauthorized assistance on this assignment.

LLM Use: ***During the preparation of this assignment, Timothy Suh used Claude AI in in code development, debugging, concept learning, and documentation to assist with templates, explain NLP and machine learning concepts, troubleshoot errors, and structure project documentation. After using this tool, I/we reviewed and edited the content as needed to ensure its accuracy and take full responsibility for the content in relation to grading.***

2. **(15pts) Learning Objectives:**
   List the learning objectives from your proposal. In your own words explain whether you met those objectives and how (50-100 words each objective). Also describe if you learned something different than expected or anything additional.

1. Understand how text classification works by learning the fundamentals of natural language processing (NLP), including tokenization, feature extraction, and training a supervised machine learning model to classify text as likely real or likely misinformation.

I successfully met this objective by creating a complete NLP pipeline from scratch. I learned how TF-IDF (Term Frequency-Inverse Document Frequency) converts raw text into numbers that machine learning models can process. During the implementation, I found that preprocessing steps like removing stop words and using n-grams significantly impacted the model performance. I also learned about the bias-variance tradeoff when tuning the max_features parameter. The most surprising thing that I learned that I did not expect was logistic regression being able to effectively handle high-dimensional sparse matrices from text data.

2. Develop practical skills in Python, scikit-learn, pandas, and a lightweight web framework to build and deploy a simple machine learning model. Improve my ability to integrate front-end input forms with backend inference logic, creating a functional full-stack ML app.

I exceeded this objective by building a complete Flask web application with a professional UI. I learned how to serialize ML models using joblib, load them in a web server, and handle real-time predictions. The using HTML forms and Flask routes taught me about POST requests and template rendering. I also developed skills in error handling, input validation, and creating both a web interface and a REST API endpoint.

The most challenging part was understanding how to maintain the same preprocessing pipeline between training and interference to avoid feature mismatch errors.

3. **(15pts) Timeline:**
   Outline how you spent time on your project. Break down the time into specific tasks or milestones. Here is an adjustable schedule to get you started. Actual Details should be 50-100 words each and should compare or reflect on differences from your proposal.

Time	**Task**	**Expected Details from Proposal**	**Actual Details**
Hour 1-2	Research and gather resources	I will look for publicly available datasets that contain fake and real news. I will then read documentation for scikit-learn's TF-IDF vectorizer and logistic regression classifier to understand how text classification works. I'll also review basic Flask examples to see how ML models can be exposed through a web interface. I will gather data, identify relevant features, and plan my model testing strategy.	I downloaded the Kaggle Fake News dataset and explored its structure. Using pandas, I loaded and cleaned both Fake and True datasets, removed null values, and combined the title and text columns into a single content field. I labeled fake articles as 1 and real ones as 0. This standardized datasets was then saved in the data/ folder to ensure consistent input for model training.
Hour 3-4	Design the project structure and plan	I will plan the directory structure, including folders for data, model training scripts, and the web app. I will outline the model process from	I wrote a Python setup script to automatically create the full project structure, including directories for data, models, templates, static files, and notebooks. The script also generated a requirements.txt

Time	Task	Expected Details from Proposal	Actual Details
		reading and processing the data to testing the model. For the web interface, I will design a simple text entry form and a results page showing the model's prediction. I will ensure that my plan supports both the MVP and potential stretch features like confidence scoring.	with all dependencies and a README.md containing setup instructions. This automated configuration ensured that the project could be easily replicated or deployed on other systems with minimal setup effort.
Hour 5-6	Start coding the basic functionalities	I will implement the core ML pipeline: load the dataset, clean and preprocess the text, extract features using TF-IDF, and train a simple logisitic regression classifier. I will test the model using accuracy and a confusion matrix. Once trained, I will save the model and vectorizer to disk for later use in the web interface. By the end of this stage, I should have a functioning text classifier that runs locally and can predict on sample inputs.	I implemented train_model.py to train a misinformation classifier using scikit-learn's TfidfVectorizer and LogisticRegression. The script split the data into training and test sets, created TF-IDF features, trained the model, and printed evaluation metrics including accuracy, confusion matrix, and classification report. After achieving strong accuracy on the test set, I saved both the trained model and vectorizer as .pkl files in the models/ directory using joblib.

Time	Task	Expected Details from Proposal	Actual Details
Hour 7-8	Test and debug the initial version	I will test the model on new text samples to ensure predictions are reasonable. I will validate that preprocessing steps are applied correctly and fix issues such as overfitting or poor feature extraction. Then I will integrate the model into a Flask app, verifying that text entered through the UI is processed and classified properly. Debugging may involve fixing routing issues, formatting problems, or vectorization mismatches between training and inference.	I built a Flask web app that loads the trained model and vectorizer to classify user-submitted text as “Likely Real” or “Likely Fake.” The app includes form validation, confidence scoring, and probability output for each class. I designed HTML templates to display the predictions clearly. The web interface allowed real-time model interaction and provided intuititive, educational feedback about the classification.
Hour 9-10	Refine and add advanced features	I will polish the interface, adding clear prediction messages and optional confidence scores. If time allows, I will implement simple interpretability such as highlighting important words using model coefficients. I may also add input validation or basic styling to make the UI more user-friendly. Finally, I will document the project, clean up the codebase,	I developed a testing script to verify model reliability. The tests covered empty inputs, long and short texts, numeric and special character inputs, and checked that probability outputs summed to one. I also confirmed consistent predictions and confidence calculations across repeated runs. After all tests passed, I finalized documentation and verified that the system ran smoothly end-to-end.

Time	Task	Expected Details from Proposal	Actual Details
		and prepare a short explanation of how the model works and its limitations.	
Additional			

4. **(55pts) Final Product Description:**
   Include your proposed MVP, Target, and Reach versions.
   i. Minimum Viable Product (MVP):

A simple machine learning model that loads a small misinformation dataset, preprocesses text using TF-IDF, and trains a binary classifier to distinguish between real and fake news or statements. The MVP includes a section where users can input text and receive a classification result.

ii. Target Product:

A functional web app that allows users to paste text into a form and receive a prediction indicating whether the content is likely misinformation. The app runs a trained ML model on the backend using Flask and displays clear results, including model confidence. This version demonstrates full-stack integration, applied machine learning, and a clean user experience accessible in a browser.

iii. Reach Version:

An improved version with explainability features such as showing which words influenced the model's prediction. Additional features may include multiple classification categories, sentiment detection, or a "clickbait socre." The reach version may also allow uploading short text files or headlines and include visualization such as word clouds.

iv. (20pts) Description of final product including target audience, user story, problem statement, key features, technical details and technologies used. (100 – 150 words)

This product targets students, educators, and general users who are interested in learning how misinformation detection works.

As technology improves, fake news online increases, so this problem is trying to address the growing need for educational tools that can help people understand how we can identify potentially misleading content.

The key features of this product are text classification, confidence scoring with probability distribution, a clean web interface, real-time prediction, and REST API endpoint for automatic access.

A user visits the website, pastes a news headline or article, clicks “Analyze Text” and receives an immediate prediction with confidence scores showing he likelihood of the content being real or fake news.

v. (20pts) Provide a YouTube link to your video demonstration (1–2 minutes, narrated). **Important Note:** Do not upload your video file directly. Instead, upload your video to YouTube and include the video link clearly here in your report. The level of difficulty and detail of the project should be reasonable for 10 hours of work with LLM support. The project should not be something an LLM can solve without significant effort by the developer.
(Be sure to have someone else test that your link is working.)
https://youtu.be/NQ1UVv8U9Sg

vi. (15pts) Any input files, coding files, and test files should be uploaded. Provide a list here of file names and purposes, or any links to live sites or artifacts. Remember code should also be commented. A README file should be created and uploaded so that we have the option to follow your instructions to run your project.

ReadME.md – Complete setup instructions, project overview, and usage guide
Requirements.txt – List of all Python dependencies needed to run the project
Fake.csv – Fake news articles dataset from Kaggle
True.csv – Real news articles dataset from Kaggle
Classifier.pkl – Trained logistic regression classifier saved using joblib
Vectorizer.pkl – Fitted TF-IDF vectorizer for text feature extraction
Setup.py – Script to create project directory structure
App.py – Flask web application that loads model and handles predictions
Train_model.py – Trains ML model, evaluates performance, saves model and vectorizer
Test.py - Tests for model reliability, edge cases, and validation
Index.html – Main web interface with text input box
Result.html – Displays prediction results with confidence scores

Project Repository & Code Submission Details:
Project Repository (code.vt: Youhttps://code.vt.edu/ or (Git-hub only if you are part of virtual global collaboration)): r repository should be well-organized, documented, and easy to navigate. At a minimum, include the following structure:

- **code/** – All source code files for your project (organized by component or module if applicable).
- **data/** – Any input files, datasets, or configuration files used by your program.
- **tests/** – Test scripts or files demonstrating how your code was verified.
- **docs/** – Supporting materials such as screenshots, reports, or documentation.

- **report/** – This final report document.
- **README.md** – A detailed file describing:
  - Project overview and purpose
  - Video link of your project
  - Installation and setup instructions
  - How to run the program and reproduce results
  - Technologies or libraries used
  - Author(s) and contribution summary

Required:

- Maintain a logical directory structure, do not store all files at the root level.
- Include comments in your code to explain logic and design decisions.
- Keep your repository **private** until grades are released, then you may make it public.

Share access with the following personnel (Add them as collaborators):

GTA Name	Section	Professor
Mona Moghadampanah	83484	P. Sullivan
Yue Shen	83485	O. Emebo
Abdullah Al Noman	83486	O. Emebo
Suraj Vishwanath	83487	P. Sullivan
Juno Bartsch	91578	S. Nizamani

5. **(10pts) Consultation and Use of LLMs:**
   Each student must create a unique project but is allowed to consult with other people and use Large Language Models (LLMs). Describe how you incorporated these resources into your project:

- **Consultation Description:**
  Describe how you ended up seeking advice or feedback from peers, mentors, or online communities.

I discussed the initial project concept with a classmate to validate the feasibility of building a misinformation detector in 10 hours. They gave me feedback on the UI design and suggested adding confidence scores to make predictions more educational.

The technical implementation and coding were completed independently using LLM assistance.

- **Use of LLMs:**
  Explain how you ended up utilizing LLMs to assist with coding, debugging, learning technologies and concepts, or generating ideas.

I used Claude AI throughout the project for learning NLP concepts, generating code templates, and debugging assistance. The LLM explained TF-IDF vectorization, logistic regression, and Flask basics, which accelerated my learning. However, I made the architectural decisions and spent a lot of time debugging integration issues like feature consistency between training and inference. Dataset-specific challenges and UI design required hands-on problem-solving beyond the LLM's capabilities. Claude provided a strong foundation, the implementation required a lot of personal effort in testing and validation.